



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación
IIC2143 - Ingeniería de Software 2026-2

Tarea Introducción a *Rails*

Objetivos

- Aprender a utilizar el framework *Ruby on Rails* y *PostgreSQL*.
- Entender e implementar el concepto de una *API REST* en formato *JSON*.
- Practicar *CRUD*, asociaciones, validaciones y eliminación en cascada.

Fecha de Entrega

28 de Agosto del 2026, 23:59

Acceso al repositorio personal de la tarea: cada estudiante recibirá una invitación de GitHub a su repositorio privado. Es responsabilidad del estudiante aceptarla antes de comenzar la tarea.

ReservaDCC API

En esta tarea usted tendrá que crear una API que ayude en la administración de salas de estudio, estudiantes registrados y sus reservas por fecha y módulo. La API deberá permitir crear, consultar, actualizar y eliminar reservas, administrar salas y estudiantes, y realizar consultas que relacionen los tres modelos.

Tips:

- Realice la tarea con tiempo y pruebe cada endpoint antes de avanzar al siguiente.
- Utilice exactamente los nombres de modelos, atributos y rutas indicados en este enunciado; los tests los usarán para corregir su API.
- Al eliminar una sala o un estudiante, también deben eliminarse todas sus reservas asociadas. *Hint:* revise la opción `dependent: :destroy` de Rails en una relación *has_many*.
- El código base incluye tests públicos. Si Rails intenta sobrescribirlos al generar un modelo o controlador, use `--skip-test-framework`.

Para ejecutar la batería de tests públicos utilice `rails test` desde la raíz del proyecto.

Modelos mínimos

En esta sección se detallan los modelos mínimos que debe manejar la API, junto a sus atributos y validaciones. Todo lo señalado a continuación deberá ser implementado utilizando los mismos nombres, ya que de esta forma los atributos de los JSON de respuesta tendrán los nombres esperados por los tests.

Model Room

- **Atributos:**
 - name de tipo string
 - building de tipo string
 - capacity de tipo integer.
- **Asociaciones:** una sala puede tener múltiples reservas.
- **Validaciones:** *name* y *building* son obligatorios; *capacity* debe ser un entero estrictamente mayor que 0.
- **Consideraciones:** al eliminar una sala, deben eliminarse sus reservas asociadas, sin eliminar a los estudiantes involucrados.

Model Student

- **Atributos:**
 - name de tipo string
 - email de tipo string.
- **Asociaciones:** un estudiante puede tener múltiples reservas.
- **Validaciones:** *name* y *email* son obligatorios; *email* debe ser único.
- **Consideraciones:** al eliminar un estudiante, deben eliminarse sus reservas asociadas, sin eliminar las salas involucradas.

Model Reservation

- **Atributos:**
 - purpose de tipo string.
 - reservation_date de tipo date.
 - module_selected de tipo integer.
 - room de tipo references.
 - student de tipo references.
- **Asociaciones:** una reserva pertenece exactamente a una sala y a un estudiante existentes.
- **Validaciones:** *purpose*, *reservation_date* y *module_selected* son obligatorios. Además, *module_selected* debe ser un número entero entre 1 y 9.
- **Formato de fecha:** *reservation_date* se enviará como una fecha calendario en formato AAAA-MM-DD, por ejemplo, "2026-08-27". No se usan horas ni timestamps.
- **Referencias:** si falta *room_id* o *student_id*, la API debe responder *422 Unprocessable Content*. Si se envía uno que no corresponde a una sala o estudiante existente, debe responder *404 Not Found*.

- **Módulos:** cada reserva corresponde a un único módulo, identificado mediante *module_selected*. Para efectos de esta tarea se utilizará la siguiente equivalencia:

Módulo = Module_selected	Horario
1	08:20 a 09:30
2	09:40 a 10:50
3	11:00 a 12:10
4	12:20 a 13:30
5	14:50 a 16:00
6	16:10 a 17:20
7	17:30 a 18:40
8	18:50 a 20:00
9	20:10 a 21:20

- **JSON de reserva:** toda representación de una reserva debe incluir *id*, *purpose*, *reservation_date*, *module_selected*, *room_id* y *student_id*.
- **Consideraciones:** No se requiere implementar autenticación ni un endpoint de consulta de disponibilidad. Sin embargo, no se puede registrar ni actualizar una reserva si ya existe otra para la misma sala, fecha y módulo.

Rutas solicitadas

Las rutas solicitadas se dividen en tres niveles. Cada nivel contiene los endpoints respectivos y se evaluará usando tests que verifican el funcionamiento correcto de cada uno. La batería de tests tiene un máximo de 6,0 puntos; el punto restante corresponde a la documentación solicitada en el README.md.

Todas las respuestas que contengan cuerpo deben estar en formato JSON.

- Los GET y PATCH exitosos deben responder con 200 OK.
- Los POST exitosos con 201 Created.
- Los DELETE exitosos con 204 No Content y sin body.

Error de validación: 422 Unprocessable Content

Cuando una request POST o PATCH no cumpla las validaciones indicadas, la API debe responder con estado 422 Unprocessable Content y un cuerpo JSON que incluya la clave de cada atributo inválido. Por ejemplo, si se intenta crear una reserva sin *purpose* y con *module_selected* inválido, una respuesta válida sería:

```
{
  "purpose": ["can't be blank"],
  "module_selected": ["must be an integer"]
}
```

Los tests verificarán el estado 422, que la respuesta sea un JSON válido y que incluya la clave del atributo inválido. **No se exigirá un mensaje de error específico.**

Además, no puede existir más de una reserva con la misma combinación de *room_id*, *reservation_date* y *module_selected*. Si se intenta crear o actualizar una reserva duplicada, la API debe responder con 422 Unprocessable Content y el JSON de errores debe incluir la clave *room_id*. Por ejemplo, una respuesta válida sería:

```
{
  "room_id": ["already has a reservation for this date and module"]
}
```

Recurso inexistente: 404 Not Found

Cuando se solicite o se indique un recurso que no existe, la API debe responder con estado 404 Not Found y un cuerpo JSON que informe el error.

Esto incluye:

- El recurso principal indicado en la URL, por ejemplo GET /rooms/999.
- Una sala o estudiante inexistente indicado en una ruta anidada.
- Un *room_id* o *student_id* inexistente al crear o actualizar una reserva.

Por ejemplo, una respuesta válida sería:

```
{
  "error": "Room not found"
}
```

Los tests verificarán el estado 404, que la respuesta sea un JSON válido y que incluya la clave "error".

No se exigirá un mensaje de error específico.

Cuando una ruta retorne una colección, no se exigirá un orden específico de sus elementos.

Nivel 1: 1,5 ptos

1. **GET** /rooms · 0,20 puntos

Debe retornar todas las salas que se encuentren en la base de datos.

Ejemplo url: <http://127.0.0.1:3000/rooms>

Ejemplo de **response** body

```
[
  {
    "id": 1,
    "name": "Sala BC24",
    "building": "Ingeniería",
    "capacity": 60,
    "created_at": "2026-07-23T23:15:13.078Z",
    "updated_at": "2026-07-23T23:15:13.078Z"
  },
  {
    "id": 2,
    "name": "Sala BC23",
    "building": "Ingeniería",
    "capacity": 60,
    "created_at": "2026-07-23T23:15:47.930Z",
    "updated_at": "2026-07-23T23:15:47.930Z"
  }
]
```

2. **POST** /rooms · 0,30 puntos

Debe crear una nueva sala utilizando los atributos enviados en la request y responder con 201 Created.

Ejemplo url: <http://127.0.0.1:3000/rooms>

Ejemplo **request** body

```
{
  "room": {
    "name": "Sala BC24",
    "building": "Ingeniería",
    "capacity": 60
  }
}
```

Ejemplo **response** body

```
{
  "id": 4,
  "name": "Sala BC24",
  "building": "Ingeniería",
  "capacity": 60,
  "created_at": "2026-07-23T23:45:13.057Z",
  "updated_at": "2026-07-23T23:45:13.057Z"
}
```

3. **GET** /rooms/:id · 0,20 puntos

Debe retornar la sala cuyo id se entrega en la URL.

Ejemplo url: <http://127.0.0.1:3000/rooms/1>

Ejemplo de **response** body

```
{
  "id": 1,
```

```
"name": "Sala BC24",
"building": "Ingeniería",
"capacity": 60,
"created_at": "2026-07-23T23:15:13.078Z",
"updated_at": "2026-07-23T23:15:13.078Z"
}
```

4. **GET** /students · 0,20 puntos

Debe retornar todos los estudiantes que se encuentren en la base de datos.

Ejemplo url: <http://127.0.0.1:3000/students>

Ejemplo de **response** body

```
[
  {
    "id": 1,
    "name": "Pikachu",
    "email": "pikachu@uc.cl",
    "created_at": "2026-07-24T00:08:54.956Z",
    "updated_at": "2026-07-24T00:08:54.956Z"
  },
  {
    "id": 2,
    "name": "Bulbasaur",
    "email": "bulbasaur@uc.cl",
    "created_at": "2026-07-24T00:12:57.666Z",
    "updated_at": "2026-07-24T00:12:57.666Z"
  },
  {
    "id": 3,
    "name": "Charmander",
    "email": "charmander@uc.cl",
    "created_at": "2026-07-24T00:13:18.029Z",
    "updated_at": "2026-07-24T00:13:18.029Z"
  }
]
```

5. **POST** /students · 0,40 puntos

Debe crear un nuevo estudiante utilizando los atributos enviados en la request y responder con 201 Created. El email no puede repetirse.

Ejemplo url: <http://127.0.0.1:3000/students>

Ejemplo **request** body

```
{
  "student": {
    "name": "Pikachu",
    "email": "pikachu@uc.cl"
  }
}
```

Ejemplo de **response** body

```
{
  "id": 1,
  "name": "Pikachu",
  "email": "pikachu@uc.cl",
  "created_at": "2026-07-24T00:08:54.956Z",
  "updated_at": "2026-07-24T00:08:54.956Z"
}
```

6. **GET** /students/:id · 0,20 puntos

Debe retornar el estudiante cuyo id se entrega en la URL.

Ejemplo url: <http://127.0.0.1:3000/students/1>

Ejemplo de **response** body

```
{
  "id": 1,
  "name": "Pikachu",
  "email": "pikachu@uc.cl",
  "created_at": "2026-07-24T00:08:54.956Z",
  "updated_at": "2026-07-24T00:08:54.956Z"
}
```

Nivel 2: 3,5 ptos

7. **DELETE** /rooms/:id · 0,4 puntos

Debe eliminar la sala cuyo id se entrega en la URL y todas sus reservas asociadas. Las demás salas, reservas y estudiantes deben mantenerse sin cambios.

Ejemplo url: <http://127.0.0.1:3000/rooms/1>

No debe retornar contenido y el status debe ser 204 (No Content)

8. **DELETE** /students/:id · 0,4 puntos

Debe eliminar el estudiante cuyo id se entrega en la URL y todas sus reservas asociadas. Las demás salas, reservas y estudiantes deben mantenerse sin cambios.

Ejemplo url: <http://127.0.0.1:3000/students/1>

No debe retornar contenido y el status debe ser 204 (No Content)

9. **GET** /reservations · 0,2 puntos

Debe retornar todas las reservas que se encuentren en la base de datos.

Ejemplo url: <http://127.0.0.1:3000/reservations>

Ejemplo de **response** body

```
[
  {
    "id": 1,
    "purpose": "Preparación de I1",
    "reservation_date": "2026-08-27",
    "module_selected": 3,
    "room_id": 1,
    "student_id": 1,
    "created_at": "2026-07-24T00:24:21.236Z",
    "updated_at": "2026-07-24T00:24:21.236Z"
  },
  {
    "id": 2,
    "purpose": "Preparación de I2",
    "reservation_date": "2026-11-10",
    "module_selected": 4,
    "room_id": 1,
    "student_id": 1,
    "created_at": "2026-07-24T00:27:59.559Z",
    "updated_at": "2026-07-24T00:27:59.559Z"
  },
]
```

```

{
  "id": 3,
  "purpose": "Preparación de Examen",
  "reservation_date": "2026-11-18",
  "module_selected": 2,
  "room_id": 1,
  "student_id": 1,
  "created_at": "2026-07-24T00:28:13.242Z",
  "updated_at": "2026-07-24T00:28:13.242Z"
}
]

```

10. **POST** /reservations · 0,5 puntos

Debe crear una reserva asociada a una sala y a un estudiante existentes. Si falta un atributo o se incumple otra validación, debe responder 422 Unprocessable Content. También debe responder 422 Unprocessable Content si ya existe una reserva para la misma sala, fecha y módulo.

Si room_id o student_id no corresponde a un recurso existente, debe responder 404 Not Found.

Ejemplo url: <http://127.0.0.1:3000/reservations>

Ejemplo **request** body

```

{
  "reservation": {
    "purpose": "Preparación de I1",
    "reservation_date": "2026-08-27",
    "module_selected": 3,
    "room_id": 1,
    "student_id": 1
  }
}

```

Ejemplo **response** body

```

{
  "id": 1,
  "purpose": "Preparación de I1",
  "reservation_date": "2026-08-27",
  "module_selected": 3,
  "room_id": 1,
  "student_id": 1,
  "created_at": "2026-07-24T00:24:21.236Z",
  "updated_at": "2026-07-24T00:24:21.236Z"
}

```

11. **GET** /reservations/:id · 0,2 puntos

Debe retornar la reserva cuyo id se entrega en la URL, incluyendo id, purpose, reservation_date, module_selected, room_id y student_id.

Ejemplo url: <http://127.0.0.1:3000/reservations/1>

Ejemplo de **response** body

```

{
  "id": 1,
  "purpose": "Preparación de I1",
  "reservation_date": "2026-08-27",
  "module_selected": 3,
  "room_id": 1,
  "student_id": 1,
  "created_at": "2026-07-24T00:24:21.236Z",
  "updated_at": "2026-07-24T00:24:21.236Z"
}

```



```
}
```

12. **PATCH** /reservations/:id · 0,4 puntos

Se puede actualizar parcialmente purpose, reservation_date, module_selected, room_id y student_id. Los atributos que no se envíen deben conservar su valor actual.

Si se envía un room_id o student_id inexistente, debe responder 404 Not Found; si se incumple una validación —por ejemplo, un module_selected fuera del rango de 1 a 9— debe responder 422 Unprocessable Content. También debe responder 422 Unprocessable Content si la actualización genera una reserva duplicada para la misma sala, fecha y módulo. Debe retornar la reserva actualizada con 200 OK.

Ejemplo url: <http://127.0.0.1:3000/reservations/1>

Ejemplo **request** body

```
{
  "reservation": {
    "reservation_date": "2026-10-05",
    "module_selected": 5
  }
}
```

Ejemplo de **response** body

```
{
  "reservation_date": "2026-10-05",
  "module_selected": 5,
  "room_id": 1,
  "student_id": 1,
  "id": 1,
  "purpose": "Preparación de I1",
  "created_at": "2026-07-24T00:24:21.236Z",
  "updated_at": "2026-07-24T00:35:07.065Z"
}
```

13. **DELETE** /reservations/:id · 0,2 puntos

Debe eliminar la reserva cuyo id se entrega en la URL. No debe retornar contenido y debe conservar la sala y el estudiante asociados.

Ejemplo url: <http://127.0.0.1:3000/reservations/1>

No debe retornar contenido y el status debe ser 204 (No Content)

14. **GET** /rooms/:room_id/reservations · 0,6 puntos

Debe retornar únicamente las reservas de la sala indicada por :room_id. Si la sala no existe, debe responder 404 Not Found. Si la sala existe y no tiene reservas, debe retornar un arreglo vacío.

Ejemplo url: <http://127.0.0.1:3000/rooms/1/reservations>

Ejemplo de **response** body

```
[
  {
    "id": 1,
    "purpose": "Preparación de I1",
    "reservation_date": "2026-10-05",
    "module_selected": 5,
    "room_id": 1,
    "student_id": 1,
    "created_at": "2026-07-24T00:24:21.236Z",
  }
]
```

```

      "updated_at": "2026-07-24T00:35:07.065Z"
    },
    {
      "id": 2,
      "purpose": "Preparación de I2",
      "reservation_date": "2026-11-10",
      "module_selected": 4,
      "room_id": 1,
      "student_id": 1,
      "created_at": "2026-07-24T00:27:59.559Z",
      "updated_at": "2026-07-24T00:27:59.559Z"
    },
    {
      "id": 3,
      "purpose": "Preparación de Examen",
      "reservation_date": "2026-11-18",
      "module_selected": 2,
      "room_id": 1,
      "student_id": 1,
      "created_at": "2026-07-24T00:28:13.242Z",
      "updated_at": "2026-07-24T00:28:13.242Z"
    }
  ]
}

```

15. **GET** /students/:student_id/reservations · 0,6 puntos

Debe retornar únicamente las reservas del estudiante indicado por :student_id. Si el estudiante no existe, debe responder 404 Not Found. Si el estudiante existe y no tiene reservas, debe retornar un arreglo vacío.

Ejemplo url: <http://127.0.0.1:3000/students/2/reservations>

Ejemplo de **response** body

```

[
  {
    "id": 6,
    "purpose": "Preparación de I1",
    "reservation_date": "2026-08-27",
    "module_selected": 3,
    "room_id": 2,
    "student_id": 2,
    "created_at": "2026-07-24T00:43:46.117Z",
    "updated_at": "2026-07-24T00:43:46.117Z"
  },
  {
    "id": 7,
    "purpose": "Preparación de I2",
    "reservation_date": "2026-11-11",
    "module_selected": 3,
    "room_id": 3,
    "student_id": 2,
    "created_at": "2026-07-24T00:43:56.976Z",
    "updated_at": "2026-07-24T00:43:56.976Z"
  }
]

```

Nivel 3: 1,0 pto

16. **GET** /rooms/min_capacity/:capacity · 0,5 puntos

Debe retornar las salas cuya capacidad sea mayor o igual al valor entregado en :capacity. Si ninguna sala cumple, debe retornar un arreglo vacío.

Ejemplo url: http://127.0.0.1:3000/rooms/min_capacity/30

Ejemplo de **response** body

```
[
  {
    "id": 1,
    "name": "Sala BC24",
    "building": "Ingeniería",
    "capacity": 60,
    "created_at": "2026-07-23T23:15:13.078Z",
    "updated_at": "2026-07-23T23:15:13.078Z"
  },
  {
    "id": 2,
    "name": "Sala BC23",
    "building": "Ingeniería",
    "capacity": 60,
    "created_at": "2026-07-23T23:15:47.930Z",
    "updated_at": "2026-07-23T23:15:47.930Z"
  },
  {
    "id": 3,
    "name": "Sala A7",
    "building": "Ingeniería",
    "capacity": 80,
    "created_at": "2026-07-23T23:18:13.795Z",
    "updated_at": "2026-07-23T23:18:13.795Z"
  },
  {
    "id": 5,
    "name": "Sala B17",
    "building": "Ingeniería",
    "capacity": 40,
    "created_at": "2026-07-24T00:45:24.824Z",
    "updated_at": "2026-07-24T00:45:24.824Z"
  }
]
```

17. **GET** /students/:student_id/reservations/count · 0,5 puntos

Debe retornar un objeto JSON con el id del estudiante y la cantidad de reservas que tiene asociadas. Si el estudiante no existe, debe responder 404 Not Found; si existe y no tiene reservas, reservations_count debe ser 0.

Ejemplo url: <http://127.0.0.1:3000/students/1/reservations/count>

Ejemplo de **response** body

```
{
  "student_id": 1,
  "reservations_count": 4
}
```

Evaluación

Para evaluar que su API funciona correctamente se realizarán consultas que verificarán la respuesta y el status HTTP de cada endpoint. Cada test tiene puntaje asociado y la batería completa suma un máximo de 6,0 puntos.

Se facilitará una batería de tests públicos que podrá ejecutar desde la raíz del proyecto utilizando el comando rails test. Los tests privados verificarán los requisitos descritos en este enunciado con datos distintos. **No se evaluarán requisitos que no estén descritos en este enunciado.** La documentación del README.md se calificará separadamente por 1,0 punto, de acuerdo con los requisitos de la sección Entrega.

Entrega

La entrega de esta tarea deberá realizarse en el repositorio de GitHub que se le facilite. Es importante que su código se encuentre en la rama main; de lo contrario, la tarea no se revisará. El repositorio incluye una plantilla README.md, cuya documentación equivale a 1,0 punto de la evaluación. Para obtener este punto, se deben completar todos los ítems listados a continuación. Debe incluir su nombre, apellido, número de alumno y usuario de GitHub. El README.md debe registrar brevemente los comandos que realmente utilizó para crear y ejecutar su proyecto; no basta con dejar el README genérico de Rails.

Orden sugerido para la realización de la tarea (aparece también en el README).

1. Instalar las dependencias del proyecto con bundle install.
2. Documentar la configuración de PostgreSQL e incluir un archivo .env de ejemplo, sin utilizar credenciales reales.
3. Generar los modelos Room, Student y Reservation, no olvidar el uso de `--skip-test-framework`.
4. Ejecutar los comandos para crear y migrar la base de datos.
5. Levantar el servidor y ejecutar los tests públicos.
6. Explicar brevemente las asociaciones entre los modelos, las validaciones, el uso de reservation_date y module_selected, y la eliminación en cascada.
7. Completar también las secciones de bibliografía, declaración de uso de IA cuando corresponda y checklist de entrega. Si utilizó IA generativa, indique la herramienta, el propósito de uso y cómo revisó o modificó el resultado obtenido.

Es responsabilidad del alumno preocuparse de aceptar la invitación a su repositorio respectivo. No se aceptarán entregas fuera del repositorio entregado ni fuera de plazo. Cualquier información oficial adicional sobre la tarea será publicada como Issue en el repositorio del curso.

Ruta de ejemplo: <https://github.com/IIC2143/iic2143-tarea-2026-2-usuarioGithub>

Distribución de puntaje

- Documentación (README.md): 1,0 punto
- Batería de tests: 6,0 puntos

Puntaje total: 7,0 puntos

Política de integridad académica

Los/as estudiantes de la Escuela de Ingeniería de la Pontificia Universidad Católica de Chile deben mantener un comportamiento acorde a la Declaración de Principios de la Universidad. En particular, se espera que mantengan altos estándares de honestidad académica. Cualquier acto deshonesto o fraude académico está prohibido; los/as estudiantes que incurran en este tipo de acciones se exponen a un Procedimiento Sumario. Es responsabilidad de cada estudiante conocer y respetar el documento sobre Integridad Académica publicado por la Dirección de Docencia de la Escuela de Ingeniería.

Específicamente, para los cursos del Departamento de Ciencia de la Computación, rige obligatoriamente la siguiente *política de integridad académica*.

Todo trabajo presentado por un/a estudiante para los efectos de la evaluación de un curso debe ser hecho por el/la estudiante. El/la estudiante es **responsable de originar el material entregado y de conocer íntegramente su contenido**. Asimismo, deberá respetar estrictamente las condiciones definidas por el curso para cada trabajo (por ejemplo, si el trabajo es individual, grupal, y si se permite o no el uso de material digital o de asistentes de inteligencia artificial). Por “trabajo” se entiende en general las interrogaciones escritas, las tareas de programación u otras, los trabajos de laboratorio, los proyectos, el examen, entre otros.

Si un/a estudiante incurre en una falta a la integridad académica, como, por ejemplo, entregando trabajo que no es propio, no comprendiendo íntegramente el trabajo entregado, o no cumpliendo con las condiciones establecidas por el curso, **obtendrá nota final 1.1 en el curso** y se solicitará a la Dirección de Pregrado de la Escuela de Ingeniería que no le permita retirar el curso de la carga académica semestral. En todos los casos, se informará a la Comisión de Integridad Académica de la Escuela de Ingeniería para que tome sanciones adicionales si lo estima pertinente.

En caso de utilización de fuentes digitales como Wikipedia o el uso de asistentes inteligentes como ChatGPT o Copilot, cuando se permita su uso, debe declararse de forma explícita el material o herramienta usada. En caso de uso de asistentes de inteligencia artificial, el profesor/a del curso puede solicitar que se entreguen respaldos sobre su utilización (por ejemplo, historial o *logs*).

Lo anterior se entiende como complemento al Reglamento del Estudiante de la Pontificia Universidad Católica de Chile (<https://registrosacademicos.uc.cl/reglamentos/estudiantiles/>). Por ello, es posible pedir a la Universidad la aplicación de sanciones adicionales especificadas en dicho reglamento.